

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Направление: 02.03.01 Математика и компьютерные науки

Математическая логика и теория формальных языков
ОТЧЁТ О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ
РАБОТЫ №1

Лексический анализатор

Вариант 31

Студент,
группы 5130201/30102

_____ Михайлова А. А.

Преподаватель

_____ Востров А. В.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Математическое описание	4
1.1 Структура транслятора	4
1.2 Лексический анализ	4
1.3 Назначение лексического анализа	4
1.4 Формальная модель лексического анализатора	5
1.5 Синтаксические диаграммы	6
1.6 Обработка ошибок	8
2 Особенности реализации	9
2.1 Класс LexicalAnalyzer	9
2.1.1 tokenize	9
2.1.2 _skip_whitespace	11
2.1.3 _skip_comment	12
2.1.4 generate_table	12
2.1.5 print_errors	13
2.1.6 print_table	13
3 Результаты работы программы	14
Заключение	17
Список использованной литературы	18

Введение

Разработать лексический анализатор входного текста в соответствии с вариантом работы.

Вариант 31. Входной язык содержит только кириллицу, арифметические выражения, разделенные символом ; (точка с запятой). Арифметические выражения состоят из идентификаторов, символьных констант (один символ в одинарных кавычках), знака присваивания (:=), +=, знаков операций +, −, *, / и фигурных скобок.

Длина идентификатора и строковых констант ограничена 15 символами. Программа должна допускать наличие комментариев неограниченной длины во входном файле.

Задачи программы

1. Разбиение входного текста на лексемы;
2. Классификация лексем по типам (идентификатор, знак присваивания, символьная константа, разделитель и т.д.);
3. Проверка ограничений на длину имён идентификаторов;
4. Обработка комментариев произвольной длины;
5. Вывод содержательных сообщений об ошибках, обнаруживаемых в процессе лексического анализа текста;
6. Порождение таблицы выделенных в тексте программы лексем.

1 Математическое описание

1.1 Структура транслятора

Транслятор осуществляет преобразование цепочек языка в выходные данные, и процесс трансляции связывается с двумя основными этапами.

1. Блок-распознаватель обеспечивает лексический анализ текста и строит структуру выходной цепочки на основе порождающей грамматики входного языка.
2. Блок-генератор использует построенную структуру для генерации выхода, выражающего смысл входной цепочки.

В реальных трансляторах этапы распознавания и генерации разделены не так явно, но во всех случаях синтаксическая трансляция основана на построении структуры входной цепочки целиком или по частям.

1.2 Лексический анализ

Первая фаза трансляции — лексический анализ, который выделяет **лексемы** — минимальные единицы языка, обладающие смыслом.

- В естественном языке лексемами являются слова (словоформы).
- В языках программирования — идентификаторы, ключевые слова, константы, составные операторы (например, “:=”).

На рис. 1 представлена схема транслятора.

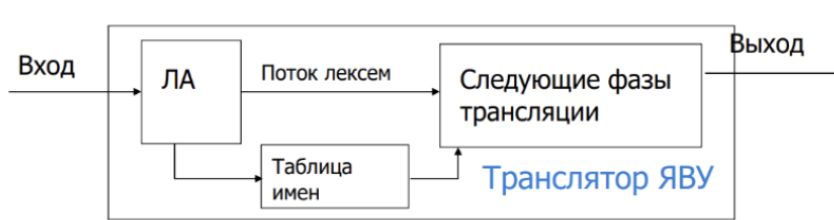


Рис. 1: Схема транслятора

1.3 Назначение лексического анализа

Лексический анализатор выполняет:

- Определение **значения лексемы** — подстроки символов, соответствующей распознанному классу. Для некоторых классов (например, чисел) значение преобразуется во внутреннее представление (двоичный формат) для компактности и ранней проверки корректности.
- Определение **класса лексемы** — категории элементов с общими свойствами:
 - идентификатор,
 - целое число,
 - строка символов,
 - оператор.

Устройство, осуществляющее сопоставление класса с каждым отдельным символом, называется **транслитератором**. Среди наиболее типичных классов символов можно привести:

- буква — класс, с которым сопоставляется множество букв (необязательно одного алфавита);
- цифра — множество символов, относящихся к цифрам;
- разделитель — пробел, перевод строки, возврат каретки;
- игнорируемый символ — символы, встречающиеся во входном потоке, но игнорирующиеся анализатором (например, служебный код);
- запрещенный символ — символы, не относящиеся к алфавиту языка, но встречающиеся во входной цепочке;
- прочие — символы, не вошедшие ни в одну из определенных категорий.

1.4 Формальная модель лексического анализатора

Пусть заданы:

- Σ — входной алфавит, задающий множество допустимых символов;
- T — множество типов токенов;
- D — множество допустимых значений токенов.

Тогда лексический анализатор реализует отображение

$$F_{lexer} : \Sigma^* \rightarrow (T \times D)^*,$$

где:

- Σ^* – множество всех возможных цепочек над Σ ;
- $(T \times D)$ – множество пар (тип токена, значение).

В данной работе:

$\Sigma = \{A, \dots, Я, a, \dots, я, 0, \dots, 9, :=, +, =, +, -, *, /, \{, \}, ;, '\}$

$T = \{i, c, z, zp, o, l, r, d\}$ – множество типов лексем, где

- i – идентификатор,
- c – символьная константа,
- z – знак присваивания,
- zp – сложение с присваиванием,
- o – оператор,
- l – левая скобка,
- r – правая скобка,
- d – разделитель.

$$D = D_{\text{ind}} \cup \{0, \dots, 9, :=, +, =, +, -, *, /, \{, \}, ;, '\},$$

где

$$D_{\text{ind}} = \{w \in \Sigma_{\text{id}}^+ \mid 1 \leq |w| \leq 15, w[1] \in \{A, \dots, Я, a, \dots, я\}, \forall i \in [2, w] : w[i] \in \Sigma_{\text{id}}\},$$

$$\text{а } \Sigma_{\text{id}} = \{A, \dots, Я, a, \dots, я, 0, \dots, 9\}.$$

1.5 Синтаксические диаграммы

Синтаксические диаграммы визуализируют правила формирования лексем.

Лексемы в данном языке программирования: идентификатор, символьные константы, знаки присваивания, операторы, левая фигурная скобка, правая фигурная скобка, разделитель.

- y_1 – символ в буффер;
- y_2 – выдать идентификатор и его номер
- y_3 – выдать символьную константу
- y_4 – выдать лексему " :="
- y_5 – выдать лексему "+"
- y_6 – выдать лексему "+="
- y_7 – выдать лексему "-"
- y_8 – выдать лексему "*"

y12 - выдать лексему ";"

На рис. 2 представлена синтаксическая диаграмма.

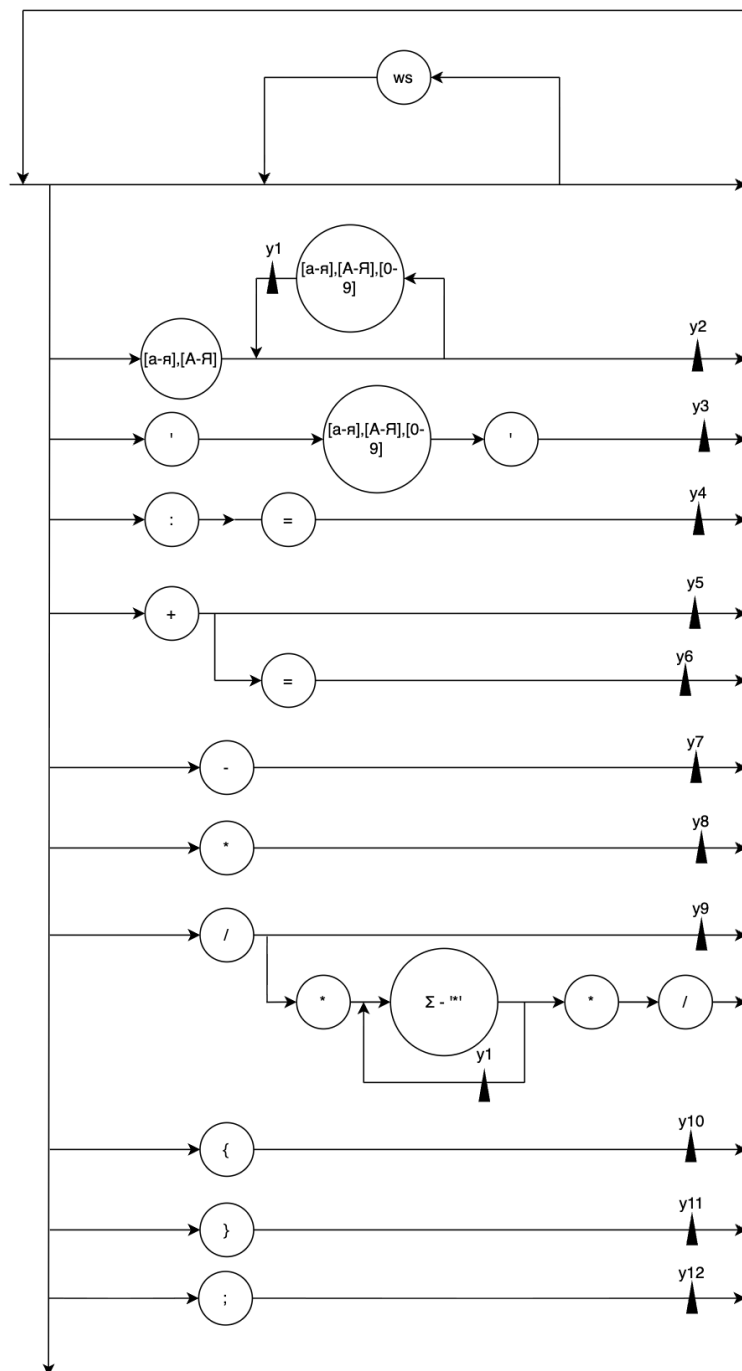


Рис. 2: Синтаксическая диаграмма

1.6 Обработка ошибок

Лексический анализатор обнаруживает следующие ошибки:

- Недопустимые символы: $\exists c \notin \Sigma$
- Превышение длины идентификатора: $|id| > 16$
- Начало имени идентификатора с цифры
- Некорректные форматы символьных констант
- Незакрытые комментарии

Для каждой ошибки генерируется сообщение с описанием и позицией ошибки.

2 Особенности реализации

Переменные, задающие типы токенов. Когда анализатор видит слово, подходящее под правило переменной, он возвращает эту метку.

```
1 TOKEN_IDENTIFIER = "IDENTIFIER"
2 TOKEN_CHAR_CONST = "CHAR_CONSTANT"
3 TOKEN_ASSIGN = "ASSIGN"
4 TOKEN_PLUS_ASSIGN = "PLUS_ASSIGN"
5 TOKEN_OPERATOR = "OPERATOR"
6 TOKEN_LBRACE = "LBRACE"
7 TOKEN_RBRACE = "RBRACE"
8 TOKEN_SEMICOLON = "SEMICOLON"
9 TOKEN_COMMENT = "COMMENT"
10 TOKEN_UNKNOWN = "UNKNOWN"
```

Правила распознавания: идентификатор должен начинаться с кириллицы и может продолжаться буквами или цифрами, если длина больше 15, анализатор должен выдать ошибку.

```
1 MAX_LEN = 15
2 CYRILLIC_LETTER = r'[А-Яа-яЁё]',
3 IDENT_REGEX = re.compile(r'^' + CYRILLIC_LETTER + r'[А-Яа-яЁё0-9]*$')
```

Словарь операторов и символов: MULTI_OPERATORS - операторы из нескольких символов, SINGLE_SYMBOLS - простая пунктуация (скобки, точки с запятой)

```
1 MULTI_OPERATORS = {':=', '+='}
2 SINGLE_OPERATORS = {'+', '-', '*', '/'}
3 SINGLE_SYMBOLS = {';', '{', '}'}
```

2.1 Класс LexicalAnalyzer

Класс LexicalAnalyzer выполняет лексический анализ входного текста и порождает таблицу лексем с указанием их типов и значений.

2.1.1 tokenize

Проходит по тексту посимвольно обрабатывая ошибки, формируя пары (тип, значение)

Вход: строка текста

Выход: пары тип + значение

```
1 def tokenize(self, text):
2     self.tokens = []
3     self.errors = []
4     self.identifier_table = {}
5     self.identifier_count = 1
```

```

6
7 length = len(text)
8 i = 0
9
10 while i < length:
11 i = self._skip_whitespace(text, i)
12 if i >= length:
13 break
14
15 if i + 1 < length and text[i:i + 2] == '/*':
16 new_i, not_closed = self._skip_comment(text, i)
17 if not_closed:
18 self.errors.append((i, "незакрытый комментарий"))
19 i = new_i
20 continue
21
22 two_ch = text[i:i + 2]
23 if two_ch in MULTI_OPERATORS:
24 if two_ch == ':=':
25 self._push_token(TOKEN_ASSIGN, two_ch)
26 elif two_ch == '+=':
27 self._push_token(TOKEN_PLUS_ASSIGN, two_ch)
28 i += 2
29 continue
30
31 ch = text[i]
32 if ch in SINGLE_OPERATORS:
33 self._push_token(TOKEN_OPERATOR, ch)
34 i += 1
35 continue
36
37 if ch in SINGLE_SYMBOLS:
38 if ch == ';':
39 self._push_token(TOKEN_SEMICOLON, ch)
40 elif ch == '{':
41 self._push_token(TOKEN_LBRACE, ch)
42 elif ch == '}':
43 self._push_token(TOKEN_RBRACE, ch)
44 i += 1
45 continue
46
47 if ch == '"':
48 end_pos = text.find('"', i + 1)
49 if end_pos == -1:
50 self.errors.append((i, "незакрытая символьная константа"))
51 i += 1
52 continue
53 char_const = text[i:end_pos + 1]
54 content_length = len(char_const) - 2
55 if content_length == 1:
56 if len(char_const) > MAX_LEN + 2:
57 self.errors.append((i, f"символьная константа длиннее {MAX_LEN} с

```

```

        ИМБОЛОВ"))
58 else:
59 self._push_token(TOKEN_CHAR_CONST, char_const)
60 else:
61 self.errors.append(
62 (i, f"СИМВОЛЬНАЯ КОНСТАНТА ДОЛЖНА СОДЕРЖАТЬ РОВНО ОДИН СИМВОЛ: {
        char_const}"))
63 i = end_pos + 1
64 continue
65
66 if ch.isdigit():
67 start_pos = i
68 while i < length and (text[i].isdigit() or re.match(
        CYRILLIC_LETTER, text[i])):
69 i += 1
70 chunk = text[start_pos:i]
71 self.errors.append((start_pos, f"НЕДОПУСТИМЫЙ ИДЕНТИФИКАТОР \"{
        chunk}\" - НЕ МОЖЕТ НАЧИНАТЬСЯ С ЦИФРЫ"))
72 continue
73
74 if re.match(CYRILLIC_LETTER, ch):
75 start_pos = i
76 while i < length and (re.match(CYRILLIC_LETTER, text[i]) or text[
        i].isdigit()):
77 i += 1
78 chunk = text[start_pos:i]
79
80 if len(chunk) > MAX_LEN:
81 self.errors.append((start_pos, f"ИДЕНТИФИКАТОР \"{chunk}\" ДЛИННЕ
        Е {MAX_LEN} СИМВОЛОВ"))
82 elif not IDENT_REGEX.match(chunk):
83 self.errors.append((start_pos, f"НЕДОПУСТИМЫЕ СИМВОЛЫ В ИДЕНТИФИК
        АТОРЕ \"{chunk}\""))
84 else:
85 self._push_token(TOKEN_IDENTIFIER, chunk)
86 continue
87
88 self.errors.append((i, f"НЕИЗВЕСТНЫЙ СИМВОЛ \"{ch}\""))
89 i += 1
90
91 return self.tokens

```

2.1.2 _skip_whitespace

Двигает указатель вперед, когда встречается пробел.

Вход: строка текста, позиция

Выход: сдвинутый указатель

```

1 def _skip_whitespace(self, text, pos):
2 while pos < len(text) and text[pos].isspace():
3 pos += 1

```

```
4 return pos
```

2.1.3 _skip_comment

Ищет пару */. Возвращает новую позицию и флаг (закрит комментарий или нет).

Вход: строка текста, позиция

Выход: новая позиция, флаг

```
1 def _skip_comment(self, text, start_pos):
2 pos = start_pos + 2
3 while pos < len(text):
4 if pos + 1 < len(text) and text[pos:pos + 2] == '*/':
5 return pos + 2, False
6 pos += 1
7 return pos, True
```

2.1.4 generate_table

Проходит по всем найденным токенам, проверяя был ли элемент уже в таблице и присваивая номер.

Вход: токены

Выход: таблица лексем с указанием типов и значений

```
1 def generate_table(self):
2 output_data = []
3
4 for ttype, tvalue in self.tokens:
5 if ttype == TOKEN_IDENTIFIER:
6 if tvalue not in self.identifier_table:
7 self.identifier_table[tvalue] = self.identifier_count
8 self.identifier_count += 1
9 output_data.append((tvalue, "идентификатор", f"{tvalue}:{self.
    identifier_table[tvalue]}"))
10
11 elif ttype == TOKEN_CHAR_CONST:
12 clean_value = tvalue[1:-1]
13 output_data.append((tvalue, "символьная константа", clean_value))
14
15 elif ttype == TOKEN_ASSIGN:
16 output_data.append((tvalue, "знак присваивания", tvalue))
17
18 elif ttype == TOKEN_PLUS_ASSIGN:
19 output_data.append((tvalue, "сложение с присваиванием", tvalue))
20
21 elif ttype == TOKEN_OPERATOR:
22 output_data.append((tvalue, "оператор", tvalue))
23
24 elif ttype == TOKEN_LBRACE:
```

```

25 output_data.append((tvalue, "левая скобка", tvalue))
26
27 elif ttype == TOKEN_RBRACE:
28     output_data.append((tvalue, "правая скобка", tvalue))
29
30 elif ttype == TOKEN_SEMICOLON:
31     output_data.append((tvalue, "разделитель", tvalue))
32
33 else:
34     output_data.append((tvalue, ttype, tvalue))
35
36 return output_data

```

2.1.5 print_errors

Выводит список всех накопленных ошибок с позициями.

Вход: список ошибок

Выход: выведенный в консоль список ошибок с позициями

```

1 def print_errors(self):
2     if self.errors:
3         print("найдена ошибка :(")
4         for pos, error_msg in self.errors:
5             print(f"позиция {pos+1}: {error_msg}")
6     else:
7         print("ошибок не обнаружено ^-^")

```

2.1.6 print_table

Выводит на экран таблицу лексем.

Вход: таблица лексем

Выход: выведенная в консоль таблица лексем

```

1 def print_table(self):
2     output_data = self.generate_table()
3
4     print("таблица лексем:")
5     print("=" * 60)
6     print(f"{'лексема':<20} {'тип лексемы':<25} {'значение':<20}")
7     print("-" * 60)
8     for lexeme, lexeme_type, value in output_data:
9         print(f"{lexeme:<20} {lexeme_type:<25} {value:<20}")
10    print("=" * 60)

```

3 Результаты работы программы

Входной текст:

```
а := 'а'; Сумм += '1'; { рез := '6' }
```

```
1 ошибок не обнаружено ^-^
2 таблица лексем:
3 =====
4 лексема                тип лексемы                значение
5 -----
6 а                      идентификатор                а:1
7 :=                    знак присваивания            :=
8 'а'                   символная константа        а
9 ;                     разделитель                ;
10 Сумм                  идентификатор                Сумм:2
11 +                     оператор                +
12 +=                    сложение с присваиванием    +=
13 '1'                   символная константа        1
14 ;                     разделитель                ;
15 {                     левая скобка                {
16 рез                   идентификатор                рез:3
17 :=                    знак присваивания            :=
18 '6'                   символная константа        6
19 }                     правая скобка                }
20 =====
```

Входной текст:

```
а := '1'; б += '2'; в := а + 5б - в * г / д1; { е ::= '3' }
```

```
1 найдена ошибка :(
2 позиция 30: недопустимый идентификатор "5б" - не может начинаться
  с цифры
3 позиция 51: неизвестный символ ":"
4 позиция 54: неизвестный символ "="
5 таблица лексем:
6 =====
7 лексема                тип лексемы                значение
8 -----
9 а                      идентификатор                а:1
10 :=                    знак присваивания            :=
11 '1'                   символная константа        1
12 ;                     разделитель                ;
13 б                     идентификатор                б:2
14 +=                    сложение с присваиванием    +=
15 '2'                   символная константа        2
16 ;                     разделитель                ;
17 в                     идентификатор                в:3
18 :=                    знак присваивания            :=
```

19	a	идентификатор	a:1
20	+	оператор	+
21	-	оператор	-
22	в	идентификатор	в:3
23	*	оператор	*
24	г	идентификатор	г:4
25	/	оператор	/
26	д1	идентификатор	д1:5
27	;	разделитель	;
28	{	левая скобка	{
29	е	идентификатор	е:6
30	:=	знак присваивания	:=
31	'3'	символьная константа	3
32	}	правая скобка	}
33	=====		

Входной текст:

a := 'a'; /* это комментарий без закрытия

```

1 найдена ошибка :(
2 позиция 11: незакрытый комментарий
3 таблица лексем:
4 =====
5 лексема          тип лексемы          значение
6 -----
7 а                идентификатор          а:1
8 :=              знак присваивания        :=
9 'а'              символьная константа    а
10 ;               разделитель          ;
11 =====

```

Входной текст:

аб := 'аб'; сумма := 'в'

```

1 найдена ошибка :(
2 позиция 7: символьная константа должна содержать ровно один символ: 'аб'
3 таблица лексем:
4 =====
5 лексема          тип лексемы          значение
6 -----
7 аб               идентификатор          аб:1
8 :=              знак присваивания        :=
9 ;               разделитель          ;
10 сумма           идентификатор          сумма:2
11 :=              знак присваивания        :=
12 'в'              символьная константа    в
13 =====

```

Входной текст:

идентификааааааааааатор := 'а'

[illegible]

Входной текст:

```
a := 'a'; c @= 1; p := 'b' '' 'a6c6calala'
```

```

1  найдена ошибка :(
2  позиция 13: неизвестный символ "@"
3  позиция 14: неизвестный символ "="
4  позиция 16: недопустимый идентификатор "1" - не может начинаться
   с цифры
5  позиция 31: символьная константа должна содержать ровно один символ:
   'a6bcalala'
6  таблица лексем:
7  =====
8  лексема                тип лексемы                значение
9  -----
10 a                       идентификатор                a:1
11 :=                      знак присваивания           :=
12 'a'                     символьная константа       a
13 ;                       разделитель                 ;
14 c                       идентификатор                c:2
15 ;                       разделитель                 ;
16 p                       идентификатор                p:3
17 :=                      знак присваивания           :=
18 '6'                     символьная константа       6
19 ''                      символьная константа
20 =====

```


Заключение

В ходе выполнения лабораторной работы был разработан лексический анализатор для языка, содержащего арифметические выражения, кириллицу, знаки присваивания и знаки операций. Язык был представлен в виде синтаксической диаграммы, был написан код на языке Python, обеспечивающий работу синтаксического анализатора.

Достоинства реализации:

- функции `_skip_whitespace` и `_skip_comment` пропускают пробелы и комментарии за один проход, избегая повторного анализа одних и тех же символов. Это экономит время при обработке больших текстов;
- использование предварительно скомпилированных регулярных выражений ускоряет проверку токенов.

Недостатки:

- для каждой позиции в тексте производится последовательная проверка всех шаблонов, пока не будет найдено совпадение. при наличии ошибки в тексте проверяются все шаблоны, что неэффективно для больших программ.

Масштабирование: расширение набора операторов и типов данных (римские числа, операторы цикла).

Список литературы

- [1] Секция «Телематика». — Текст: электронный // tema.spbstu: [сайт]. — URL: <https://tema.spbstu.ru/mathem/> (дата обращения: 27.03.2026)